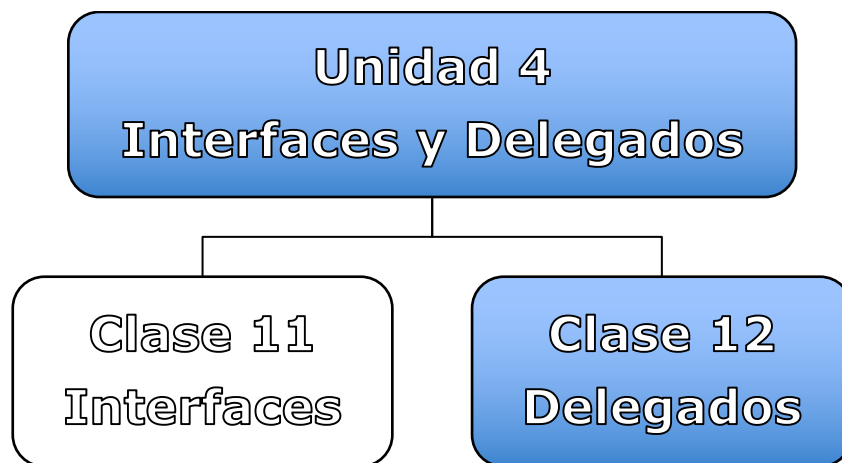

PROGRAMACIÓN ORIENTADA A OBJETOS



Docente titular y autor de contenidos: Prof. Ing. Darío Cardacci

Presentación

En esta unidad veremos todo lo necesario para poder trabajar con Interfaces y Delegados.

Esto nos permitirá construir programas más robustos y flexibles.

Por todo lo expresado a través del estudio de esta unidad, usted logrará:

- Comprender las ventajas de utilizar interfaces.
- Identificar, distinguir e implementar las interfaces IComparable, IComparer, ICloneable, IEnumerable e IEnumerator.
- Utilizar delegados en la programación para lograr asignaciones dinámicas.

Los siguientes **contenidos** conforman el marco teórico y práctico de esta unidad. A partir de ellos lograremos alcanzar el resultado de aprendizaje propuesto. En negrita encontrará lo que trabajaremos en la clase 12.

- Interfaces. Desarrollo e Implementación de una interfaz.
- La interfaz IComparable. La interfaz IComparer. La interfaz ICloneable. Las interfaces IEnumerable e IEnumerator.
- **Delegados. Delegados usando métodos con nombre. Delegados con métodos anónimos.**

A continuación, le presentamos un detalle de los contenidos y actividades que integran esta unidad. Usted deberá ir avanzando en el estudio y profundización de los diferentes temas, realizando las lecturas requeridas y elaborando las actividades propuestas, algunas de desarrollo individual y otras para resolver en colaboración con otros estudiantes y con su profesor tutor.

1. DELEGADOS

Lectura requerida

- <https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/delegates/>
- <https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/delegates/using-delegates>
- Deitel Harvey M. Y Paul J. Deitel. **Cómo programar en C#**. Segunda edición. Pearson. México 2007. Capítulos 11.

Lo/a invitamos ahora a comenzar con el estudio de los contenidos que conforman esta unidad.

2. DELEGADOS

Los **delegados** son objetos que hacen referencia a métodos. Se podrían visualizar como un puntero a una función, aunque estrictamente hablando esto último no es lo más preciso. Los delegados son muy útiles para darle al programador mayor poder cuando desea derivar dinámicamente ejecuciones de piezas de código.

Los **delegados** se pueden utilizar para muchas cosas, veremos algunos ejemplos. Cuando se crea una instancia de un **delegado**, se puede asociar con cualquier método que posea una **firma** compatible y un tipo de devolución. Luego, se puede invocar (o llamar) el método a través de la instancia de delegado.

Los delegados se utilizan para pasar métodos como argumentos a otros métodos. Los controladores de eventos no son más que métodos que se invocan a través de delegados. La siguiente línea de código muestra la declaración de un delegado:

```
public delegate int Calculo(int x, int y);
```

En el ejemplo **EJ0009**, se puede observar la declaración de un delegado, su instanciación y la llamada al delegado que provocará la llamada al método.

```
// Declaración del Delegado
public delegate string Delegado(String pTexto);
// Declaración de la variable D del tipo Delegado (Delegado es un delegate)
Delegado D;
1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    // Intanciación. Se denomina así al hecho de asignarle
    D = UsoDelDelegado;
}
1 referencia
public string UsoDelDelegado(string pTexto) { return pTexto; }
1 referencia
private void button1_Click(object sender, EventArgs e)
{
    // llamada al Delegado.
    MessageBox.Show(D(Interaction.InputBox("Ingrese un Texto: ")));
}
```

EJ0009

Los tipos de delegado se derivan de la clase **delegate**. Los tipos de delegado están sellados, esto implica que no se pueden derivar de ellos (no

se puede heredar de ellos). Como el delegado instanciado es un objeto, puede pasarse como un parámetro o asignarse a una propiedad. Esto permite que un método acepte un delegado como parámetro y llame al delegado en algún momento posterior dentro de su implementación.

El ejemplo **EJ0010** muestra como una función recibe por parámetro un tipo delegado.

```
// Declaración del Delegado
public delegate void Delegado(String pTexto);
// Declaración de la variable D del tipo Delegado (Delegado es un delegate)
Delegado D;
1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    // Intanciación. Se denomina así al hecho de asignarle
    D = UsoDelDelegado;
}
1 referencia
1 referencia
public void UsoDelDelegado(string pTexto) { MessageBox.Show(pTexto); }
1 referencia
public void RecibeDelegado(string pS,Delegado pD) { pD(pS); }
1 referencia
private void button1_Click(object sender, EventArgs e)
{
    RecibeDelegado(Interaction.InputBox("Ingrese Texto: "), D);
}
```

EJ0010

El método **RecibeDelegado** posee dos parámetros, **pS** de tipo **string** y **pD** de tipo **Delegado**. Luego en su implementación realiza: **pD(pS)**.

Lo que está haciendo es usar el delegado y pasarle el string como parámetro. La ventaja se hace evidente cuando visualizamos la posibilidad de pasar un delegado por parámetro, lo que es equivalente a pasar una función por parámetro.

Otra característica de los delegados es que pueden llamar a más de un método cuando se invocan. Esto se conoce como multicasting. Para agregar un método adicional a la lista de métodos del delegado, o sea la lista de invocación, simplemente se requiere agregar los métodos utilizando los operadores '+' o '+ ='.

El ejemplo **EJ0011** muestra como se realiza esto.

```

// Declaración del Delegado
public delegate void Delegado(String pTexto);
// Declaración de la variable D del tipo Delegado (Delegado es un delegate)
Delegado D1,D2,D3,TodosLosMetodos;
1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    // Intanciación. Se denomina así al hecho de asignarle
    D1 = Metodo1; D2 = Metodo2;D3 = Metodo3;
    TodosLosMetodos = D1 + D2;
    TodosLosMetodos += D3;
}
2 referencias
public void Metodo1(string pTexto) { MessageBox.Show("EL método 1 se ejecutó y muestra el texto: " + pTexto); }
3 referencias
public void Metodo2(string pTexto) { MessageBox.Show("EL método 2 se ejecutó y muestra el texto: " + pTexto); }
3 referencias
public void Metodo3(string pTexto) { MessageBox.Show("EL método 3 se ejecutó y muestra el texto: " + pTexto); }
1 referencia

```

EJ0011

Cuando se invoque al delegado **TodosLosMetodos** y se le pase un parámetro de tipo **string** se ejecutarán los métodos: **Metodo1**, **Metodo2** y **Metodos3**.

Si se desea quitar algún método del multicast, por ejemplos el **Metodo1**, se puede realizar de la siguiente manera.

TodosLosMetodos -= Metodo1;

Además, a los delegados se le pueden pasar funciones lambda o bien aprovecharlos de maneras muy versátiles. El ejemplo **EJ0012** muestra como hacerlo.

```

// Declaración del delegado.
delegate bool Dele(string pString);
delegate IEnumerable<string> Dele2(string[] pArray, int pLimite);

1 referencia
private void Form1_Load(object sender, EventArgs e)
1 referencia
{
    IEnumerable<string> FuncPrueba(string[] pArray, int pLimite)
    {
        return pArray.Where(n => n.Length >= pLimite);
    }
1 referencia
private void button1_Click(object sender, EventArgs e)
{
    Dele ejemplodelegado = pCadena => { return pCadena.Length > 6; };
    MessageBox.Show("La cadena ingresada mide más de 6 caracteres: " +
        ejemplodelegado(Interaction.InputBox("Ingrese el Texto:")));
}

1 referencia
private void button2_Click(object sender, EventArgs e)
{
    string[] Nombres = { "Ana", "Pedro", "Cecilia", "Guillermo", "Sol", "Ezequiel",
        "Martin", "María", "Marcelo", "Adriana" };
    Dele2 ejemplodelegado2 = FuncPrueba;
    textBox1.Clear();
    foreach (string S in ejemplodelegado2(Nombres,
        int.Parse(Interaction.InputBox("Ingrese la cantidad de caracteres mínima"))))
    {
        textBox1.Text += S + Environment.NewLine;
    }
}

```

EJ0012

Se observan dos delegados **Dele** y **Dele2**. El primero define que se recibe un string y retorna un bool.

El segundo define que recibe un array de string como primer parámetro y un entero como segundo. Retorna un `IEnumerable<string>`.

En el ejemplo, los parámetros de entrada de **Dele2** sirven para: el primero para recibir un array de nombres y el segundo para recibir un número que determina la cantidad mínima de caracteres que debe tener el nombre para ser seleccionado.

Luego a la variable **ejemplodelegado** de tipo **Dele** se le asigna una expresión lambda. Esta función anónima determina si la cantidad de caracteres de la palabra ingresada posee más de 6 caracteres y retorna un valor bool en base a esto.

La variable **ejemplodelegado2** de tipo **Dele2**, delega o apunta a la función **FuncPrueba**. La misma mediante LinQ retorna un `IEnumerable` con los nombres del array, cuando poseen una cantidad igual o superior a límite de caracteres que se ingresaron. Luego este resultado se recorre con un `foreach` para mostrarlo en una caja de texto.

